Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
Câmpus Birigui

Henrique José de Souza

Sistema Especialista Educacional para Validação de Compatibilidade de Hardware baseado em Problema de Satisfação de Restrições (CSP).

Birigui
2026

Henrique José de Souza

Sistema Especialista Educacional para Validação de Compatibilidade de Hardware baseado em Problema de Satisfação de Restrições (CSP).

Trabalho de Conclusão de Curso apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, como requisito parcial para conclusão do curso de Engenharia da Computação.

Área de Concentração: Área de Concentração do Trabalho

Orientador: Profa. Dra. Helen de Freitas Santos

Birigui

2026

FICHA CATALOGRÁFICA (obrigatória; impressa no verso da folha de rosto, não conta na paginação)

Rissi, Viviane Renata Ventura.
A importância da biblioteca : análise dos últimos 7 anos / Viviane R. V. Rissi, 2019
90 f. : il.

Orientador: Nome direto

Monografia (Graduação)- Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Birigui, 2019.

1. Leitura (Ensino superior). 2. Livros (Investimento). 3. Leitura – Meios auxiliares. I. Instituto Federal de Educação, Ciência e Tecnologia. II. Título.

Ficha catalográfica elaborada pela Biblioteca do IFSP – Campus Birigui

ERRATA

ERRATA (OPCIONAL). Aponta erros ortográficos contidos no trabalho, que não foram corrigidos antes da impressão, da seguinte forma: constrói-se uma pequena tabela composta pelas colunas “Folha”, “Linha”, “Onde se lê”, e “Leia-se” e aponta-se nessas colunas a localização do erro, o erro propriamente dito e sua respectiva correção. Encabeçando a errata, deve-se especificar a referência do trabalho.

FOLHA DE APROVAÇÃO (obrigatório)

NOME DO AUTOR DO TRABALHO

(Caso seja mais de um aluno, colocar em ordem alfabética)

TÍTULO DO TRABALHO: (TAMANHO 14)

Subtítulo do Trabalho, se Houver (tamanho 12)

Trabalho de Conclusão de Curso apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Câmpus Birigui, como requisito parcial para conclusão do curso de XXXX.

Orientador:
Co-orientador: (se houver)

Banca examinadora

_____

Membro 1, titulação e instituição

_____

Membro 2, titulação e instituição

_____

Membro 3, titulação e instituição

Birigui, ____ de _____ de _____.

Dedicatória (Opcional). Não digite a palavra Dedicatória. Texto no qual o autor do trabalho oferece homenagem ou dedica o seu trabalho a alguém (não usar ponto final)

AGRADECIMENTOS

Agradecimento (opcional). Folha que contém manifestação de reconhecimento a pessoas e/ou instituições que realmente contribuíram com o autor, devendo ser expressos de maneira simples.

Epígrafe (Opcional). Pensamentos retirados de um livro, uma música, um poema, normalmente relacionado ao tema do trabalho, seguida de indicação de autoria. As epígrafes podem ser colocadas também nas folhas de abertura de cada capítulo.

"*Any fool can write code that a computer can understand. Good programmers write code that humans can understand*".

Martin Fowler

RESUMO

Elemento obrigatório, constituído de uma sequência de frases concisas e objetivas, fornecendo uma visão rápida e clara do conteúdo do estudo. O texto deverá conter entre 150 a 250 palavras e ser antecedido pela referência do estudo. Também, não deve conter citações e deverá ressaltar o objetivo, o método, os resultados e as conclusões. O resumo deve ser redigido em parágrafo único, seguido das palavras representativas do conteúdo do estudo, isto é, palavras-chave, em número de três a cinco, separadas entre si por ponto e finalizadas também por ponto. Usar o verbo na terceira pessoa do singular, com linguagem impessoal (pronome SE), bem como fazer uso, preferencialmente, da voz ativa.

Palavras-chave: Palavra-chave 1. Palavra-chave 2. Palavra-chave 3. Palavra-chave n.

ABSTRACT OU RÉSUMÉ OU RESUMEN

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Keywords: Keyword 1. Keyword 2. Keyword 3. Keyword n.

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Mots-clés : Mot 1. Mot 2. Mot 3. Mot n.

Elemento obrigatório. É a versão do resumo em português para o idioma de divulgação internacional. Deve ser antecedido pela referência do estudo.

Palabras clave: Palabra 1. Palabra 2. Palabra 3. Palabra n.

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de figura 22
Figura 2 – Diagrama de Casos de Uso 31
Figura 3 – Modelo de Domínio 34
Figura 4 – Diagrama de Atividades 36
Figura 5 – Diagrama de Estados 36
Figura 6 – Diagrama de Sequência 40
Figura 7 – Diferentes Detalhamentos dos Serviços Relacionados ao Cliente 41
Figura 8 – Exemplo de Caso de Teste Elaborado na Ferramenta Testlink 45

LISTA DE QUADROS

Quadro 1 – Exemplo de quadro 23

LISTA DE TABELAS

Tabela 1 – Exemplo de tabela de 2 colunas 23
Tabela 2 – Requisitos Funcionais 29
Tabela 3 – Requisitos Não-Funcionais 30
Tabela 4 – Modelo para Especificação de Casos de Uso 32
Tabela 5 – Convenção para Nome dos Objetos no Banco de Dados 38
Tabela 6 – Tabelas Identificadas neste Trabalho 38
Tabela 7 – Estado 38
Tabela 8 – Cidade 39
Tabela 9 – NomeDaTabelaNN 39

LISTA DE ABREVIATURAS E SIGLAS

1D	Uma dimensão
2D	Duas dimensões
3D	Três dimensões

LISTA DE SÍMBOLOS

α	Letra grega minúscula Alfa
β	Letra grega minúscula Beta

SUMÁRIO

1 INTRODUÇÃO . . . 18
1.1 Justificativa . . . 18
1.2 Objetivos . . . 18
1.2.1 Objetivo Geral . . . 18
1.2.2 Objetivos Específicos . . . 18
1.3 Problema (Hipótese ou Questão de Pesquisa) . . . 18
1.4 Organização Deste Trabalho . . . 18

2 REVISÃO DA LITERATURA . . . 19
2.1 Problemas de Satisfação de Restrições (CSP) . . . 19
2.2 Complexidade Computacional . . . 20
2.3 Sistemas Especialistas . . . 21
2.4 Sistemas Tutores Inteligentes . . . 22

3 MATERIAIS E MÉTODOS . . . 27

4 DESCRIÇÃO GERAL DO PRODUTO . . . 28

5 ELICITAÇÃO DE REQUISITOS E ANÁLISE . . . 29
5.1 Requisitos do Usuário . . . 29
5.2 Requisitos do Sistema . . . 29
5.2.1 Requisitos Funcionais . . . 29
5.2.2 Requisitos Não-Funcionais . . . 30
5.2.3 Restrições, Suposições e Dependências . . . 30
5.2.4 Requisitos Adiados . . . 30
5.3 Casos de Uso . . . 30
5.3.1 Diagrama de Casos de Uso . . . 30
5.3.2 Especificação dos Casos de Uso . . . 31
5.4 Modelo de Domínio . . . 32
5.5 Diagrama de Objetos . . . 35
5.6 Diagrama de Classes de Análise . . . 35
5.7 Diagrama de Atividades . . . 35
5.8 Diagrama de Estados . . . 36

6 PROJETO DE SOFTWARE . . . 37
6.1 Projeto de Interface . . . 37
6.2 Projeto de Dados . . . 37

6.2.1 Mapeamento Objeto-Relacional 37
6.2.2 Estrutura das Tabelas no Banco de Dados 37
6.2.3 Diagrama de Pacotes 39
6.2.4 Diagrama de Classes de Projeto 39
6.3 Projeto Procedimental 39
6.3.1 Diagrama de Sequência 39
6.3.2 Diagrama de Atividades 40
6.3.3 Diagrama de Estados 40
6.4 Projeto Arquitetural 40

7 IMPLEMENTAÇÃO 42

8 TESTE 43

9 IMPLANTAÇÃO 46

10 RESULTADOS E DISCUSSÃO 47

11 TRABALHOS RELACIONADOS 48

12 CONCLUSÕES/CONCLUSÕES PARCIAIS 49

13 CRONOGRAMA 50

REFERÊNCIAS 51

APÊNDICE A – TÍTULO DO APÊNDICE A 52

APÊNDICE B – TÍTULO DO APÊNDICE B 53

APÊNDICE C – TÍTULO DO APÊNDICE C 54

APÊNDICE D – TÍTULO DO APÊNDICE D 55

APÊNDICE E – TÍTULO DO APÊNDICE E 56

ANEXO A – TÍTULO DO ANEXO A 57

ANEXO B – TÍTULO DO ANEXO B 58

ANEXO C – TÍTULO DO ANEXO C 59

ANEXO D – TÍTULO DO ANEXO D 60

ANEXO E – TÍTULO DO ANEXO E 61

1 INTRODUÇÃO

Aqui deve-se introduzir o trabalho. Trata-se da parte inicial do texto, onde devem constar: a delimitação do assunto tratado, os objetivos da pesquisa e outros elementos necessários para situar o tema do trabalho.

1.1 Justificativa

Texto da justificativa. Conte como se chegou ao "questionamento", como se identificou o problema, por que e de onde ele surgiu. É a explicação do porquê a inquietação em questão existe.

1.2 Objetivos

Descrever o que se pretende alcançar ao final do projeto.

1.2.1 Objetivo Geral

Qual seu objetivo geral.

1.2.2 Objetivos Específicos

- Objetivo específico 1;
- Objetivo específico 2;
- Objetivo específico n.

1.3 Problema (Hipótese ou Questão de Pesquisa)

Escolha uma dentre as 3 formas de escrever: 1) o problema que o trabalho propõe resolver; ou 2) as hipóteses que se deseja comprovar (ou não) com o estudo; ou 3) as questões de pesquisa levantadas para serem investigadas.

1.4 Organização Deste Trabalho

Como seu trabalho está organizado (capítulos) após a introdução.

2 REVISÃO DA LITERATURA

2.1 Problemas de Satisfação de Restrições (CSP)

Um Problema de Satisfação de Restrições (CSP) é formalmente caracterizado por uma tripla $\langle X, D, C \rangle$, na qual $X = \{X_1, \ldots, X_n\}$ denota um conjunto finito de variáveis, $D = \{D_1, \ldots, D_n\}$ representa os respectivos domínios, de modo que cada D_i delimita os valores admissíveis para X_i, e C reúne as restrições que impõem condições sobre as combinações válidas de valores (RUSSELL; NORVIG, 2010). Cada restrição C_j é expressa como um par $\langle scope, rel \rangle$, em que *scope* é uma tupla de variáveis participantes e *rel* define os valores que essas variáveis podem assumir conjuntamente (RUSSELL; NORVIG, 2010). Diz-se que o problema está resolvido quando se obtém uma atribuição completa e consistente: uma função que associa a cada variável um valor de seu domínio sem infringir nenhuma restrição de C.

A aplicação dessa formalização ao domínio de compatibilidade de hardware é direta e natural. As categorias de componentes, como processador, placa-mãe, memória RAM, GPU e fonte de alimentação, constituem o conjunto de variáveis X; os modelos comercialmente disponíveis em cada categoria formam os respectivos domínios D; e o conjunto de restrições C encapsula as regras físicas e elétricas que governam a compatibilidade entre esses componentes, tais como a conformidade de soquete entre processador e placa-mãe, o padrão de memória suportado e a capacidade energética exigida. Essa correspondência torna o CSP não apenas uma metáfora conveniente, mas o arcabouço formal mais adequado para modelar o problema de montagem e validação de configurações de hardware.

A principal distinção entre o CSP e as abordagens clássicas de busca em espaço de estados reside na representação fatorada que aquele adota: em vez de tratar o estado como uma entidade atômica e indivisível, o CSP expõe a estrutura interna do problema por meio de variáveis e restrições explícitas, permitindo modelá-lo como um grafo de restrições no qual os vértices correspondem às variáveis e as arestas codificam restrições binárias entre pares de variáveis (RUSSELL; NORVIG, 2010; KUMAR, 1992). A resolução por força bruta implica enumerar todas as combinações possíveis de valores para as variáveis, o que, em instâncias com n variáveis e domínios de tamanho d, gera um espaço de d^n atribuições candidatas (RUSSELL; NORVIG, 2010). Esse crescimento exponencial torna o algoritmo de *backtracking* puro inviável à medida que o problema escala, demandando técnicas complementares de poda e simplificação de domínio, uma vez que a resolução de um CSP genérico pertence à classe NP-completo (SIPSER, 2012).

A principal técnica para contornar a explosão combinatória é a propagação de

restrições, realizada por algoritmos de consistência. O mais consolidado na literatura é o AC-3 (*Arc Consistency Algorithm 3*), proposto por Mackworth (1977), cujo objetivo é garantir a consistência de arco em redes de restrições binárias (KUMAR, 1992). O algoritmo mantém uma fila de arcos a examinar e, para cada par de variáveis relacionadas, remove do domínio da variável os valores que não encontram suporte em nenhum valor do domínio da variável vizinha; cada remoção dispara a reinserção dos arcos afetados na fila, propagando a mudança pela rede até o ponto de estabilidade, em que nenhuma redução adicional é possível (RUSSELL; NORVIG, 2010). Assumindo um CSP com n variáveis, domínios de tamanho no máximo d e c restrições binárias, a complexidade de pior caso do AC-3 é $O(cd^3)$ (RUSSELL; NORVIG, 2010). Aplicado como pré-processamento ou intercalado ao *backtracking*, o AC-3 identifica antecipadamente inconsistências locais que, não tratadas, provocariam falhas reiteradas durante a busca global, reduzindo de forma expressiva o espaço efetivo de soluções a explorar.

2.2 Complexidade Computacional

A teoria da complexidade computacional ocupa-se da classificação de problemas de decisão segundo os recursos algorítmicos, principalmente tempo de execução e espaço de memória, exigidos para sua resolução (CORMEN et al., 2001). Dentro desse arcabouço, a classe P reúne os problemas para os quais existem algoritmos determinísticos de tempo polinomial, considerados eficientes e tratáveis mesmo para entradas de grande escala. A classe NP, por sua vez, abrange os problemas cujas soluções propostas podem ser *verificadas* em tempo polinomial por uma máquina determinística, ainda que não se disponha, até o presente, de algoritmos igualmente eficientes para *encontrá-las* (SIPSER, 2012). A questão sobre a relação entre essas duas classes, isto é, se P = NP, permanece em aberto e constitui o problema não resolvido de maior relevância para a ciência da computação teórica, razão pela qual o *Clay Mathematics Institute* o incluiu entre os sete Problemas do Milênio (SIPSER, 2012).

O interior da classe NP não é uniforme. A literatura demonstra a existência de subclasses com diferentes graus de dificuldade, incluindo problemas resolúveis em tempo subexponencial por meio de não-determinismo limitado (PAPADIMITRIOU; YANNAKAKIS, 1996). O CSP, contudo, não se enquadra nessas classes intermediárias: situa-se no topo da hierarquia como um problema NP-completo (SIPSER, 2012). Um problema é classificado como NP-completo quando pertence a NP e todo problema de NP se reduz a ele em tempo polinomial; essa classificação pode ser estabelecida por redução polinomial a partir do Problema da Satisfabilidade Booleana (SAT), um dos problemas canônicos de NP-completude (CORMEN et al., 2001; SIPSER, 2012). Consequentemente, se qualquer problema NP-completo admitisse solução em tempo polinomial, toda a classe NP colapsaria em P (SIPSER, 2012).

Essa natureza NP-completa impõe restrições severas à arquitetura de qualquer sistema de verificação de compatibilidade de hardware fundamentado em CSP. A primeira limitação é de ordem temporal: a avaliação exaustiva de todas as combinações interdependentes de componentes é inviável na prática em razão da explosão combinatória intrínseca à classe (BORODIN; CASHMAN; MAGEN, 2011). A segunda limitação é de ordem espacial: estratégias alternativas baseadas no mapeamento estático e exaustivo de regras lógicas condicionais, codificando explicitamente cada restrição de hardware por meio de estruturas *if-else* ou tabelas de decisão, não contornam o problema, pois a representação de funções lógicas complexas por árvores de decisão pode exigir espaço de tamanho exponencial, conforme demonstrado por (JUKNA et al., 1999). Diante dessas limitações, a modelagem do problema de montagem e validação de computadores exige a adoção de um motor de inferência baseado em CSP que aplique técnicas de propagação de restrições: em vez de enumerar cegamente o espaço de soluções ou armazenar um volume impraticável de regras estáticas, o motor poda dinamicamente os domínios das variáveis a cada interação do usuário, reduzindo o espaço de busca de forma incremental e viabilizando respostas em tempo real.

2.3 Sistemas Especialistas

Um Sistema Especialista (SE) é um programa de computador projetado para simular o raciocínio de um especialista humano em um domínio restrito e bem definido (RUSSELL; NORVIG, 2010). Sua arquitetura é composta por três elementos centrais: a base de conhecimento, que armazena fatos e regras sobre o domínio; o motor de inferência, que aplica essas regras aos fatos disponíveis para derivar conclusões; e a interface de explicação, que permite ao sistema justificar suas respostas ao usuário (RUSSELL; NORVIG, 2010). O motor de inferência pode operar segundo encadeamento progressivo (*forward chaining*), partindo dos fatos conhecidos em direção a conclusões, ou encadeamento regressivo (*backward chaining*), partindo de uma hipótese e verificando se os fatos disponíveis a sustentam (RUSSELL; NORVIG, 2010).

O sistema proposto neste trabalho se enquadra na categoria de sistemas especialistas baseados em restrições. A base de conhecimento é constituída pelas especificações técnicas dos componentes de hardware e pelas regras de compatibilidade que os relacionam; o motor de inferência é implementado por um algoritmo de propagação de restrições que opera sobre o grafo de CSP; e a interface apresenta ao usuário não apenas os componentes compatíveis, mas também as razões pelas quais determinadas combinações são inviáveis. Essa capacidade de explicação é uma característica fundamental dos sistemas especialistas que os distingue de simples mecanismos de filtragem: enquanto um filtro se limita a apresentar resultados, um SE fundamentado em CSP é capaz de indicar qual restrição foi violada e por que, tornando o processo de montagem educativo e transparente.

2.4 Sistemas Tutores Inteligentes

Um Sistema Tutor Inteligente (STI) é um ambiente computacional que oferece instrução personalizada e adaptada ao perfil do aprendiz, sem a necessidade de intervenção humana direta (RUSSELL; NORVIG, 2010). Diferentemente de sistemas de *e-learning* tradicionais, que apresentam o mesmo conteúdo a todos os usuários, um STI mantém modelos do estudante, do domínio e da estratégia pedagógica, ajustando dinamicamente o nível de dificuldade, o tipo de *feedback* e a sequência de conteúdos conforme o desempenho observado (RUSSELL; NORVIG, 2010). Essa personalização é o que confere ao STI sua característica de inteligência: a capacidade de diagnosticar lacunas de conhecimento e de intervir de forma contextualmente adequada.

A distinção fundamental entre um sistema de filtragem e um sistema tutor reside precisamente na natureza do *feedback* fornecido. Um mecanismo de filtragem tradicional responde à pergunta "quais componentes são compatíveis?", apresentando uma lista de resultados sem qualquer elaboração. Um STI, por sua vez, responde à pergunta "por que esses componentes são ou não compatíveis?", articulando as razões subjacentes à compatibilidade ou incompatibilidade em linguagem acessível ao aprendiz (RUSSELL; NORVIG, 2010). No contexto deste trabalho, o motor CSP cumpre o papel do modelo de domínio do STI: ao propagar restrições e podar domínios de forma incremental, ele não apenas determina quais configurações são válidas, mas também fundamenta as explicações que serão apresentadas ao usuário, tornando cada interação uma oportunidade de aprendizagem sobre os princípios técnicos que regem a montagem de computadores.

Texto da revisão da literatura, dividido em seções e subseções.

Este é um exemplo de como usar figuras. Referência cruzada: Figura 1

Figura 1 – Exemplo de figura

Fonte: Elaborada pelo autor

Este é um exemplo de como usar tabelas. Referência cruzada: Tabela 1

Tabela 1 – Exemplo de tabela de 2 colunas

Coluna 1	**Coluna 2**
Dado 1a	Dado 2a
Dado 1b	Dado 2b
Dado 1c	Dado 2c
Dado 1d	Dado 2d

Fonte: Elaborada pelo autor

Este é um exemplo de como usar quadros. Referência cruzada: Quadro 1

Quadro 1 – Exemplo de quadro

Cores		
Nome	**Hexa**	**Amostra**
Preto	#000000	
Marrom	#993300	
Vermelho	#FF0000	
Laranja	#FF3300	
Amarelo	#FFFF00	
Branco	#FFFFFF	

Fonte: Elaborada pelo autor

Este é um exemplo de como usar equações. Referência cruzada: Equação 2.1

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \tag{2.1}$$

Exemplo de inserção de lista de código fonte (**não use acentos no código!**):

```
/**
 * Classe de exemplo.
 *
 * @author David Buzatto
 */
public class Grafo {

    public static void main( String[] args ) {

        System.out.println( "Exemplo de codigo fonte!" );
        System.out.println( "Nao use acentos!" );

    }

}
```

Este é um exemplo de como inserir texto sem formatação (ambiente verbatim):

```
Texto sem formatação, como espaçamento igual.
```

Exemplo de lista de itens:

- **Item 1:** texto...;
- **Item 2:** texto...;
  - **Subitem:** texto...;
  - **Subitem:** texto...;
  - **Subitem:** texto...;
- **Item 3:** texto...;
- **Item n:** texto....

Exemplo de lista numerada:

1. **Item:** texto...;
2. **Item:** texto...;
   a) **Subitem:** texto...;
   b) **Subitem:** texto...;
   c) **Subitem:** texto...;
3. **Item:** texto...;
4. **Item:** texto....

Exemplos de comandos para texto e referências:

- Para iniciar um novo parágrafo, basta deixar uma linha em branco no código fonte;
- Não force o compilador a pular mais de uma linha, pois terá influência negativa na composição do documento;
- Sempre deixe o LaTeX realizar a formatação de parágrafos e posicionamento de elementos;
- Utilização de aspas simples (abertura `‘`, fechamento `’`): ‘Texto entre aspas simples’;

- Utilização de aspas duplas (abertura `‘‘`, fechamento `’’`): “Texto entre aspas duplas”;
- Negrito (comando `\textbf`): **texto em negrito**;
- Itálico (comando `\textit`): *texto em itálico*;
- Sublinhado (comando `\underline`): texto sublinhado;
- Negrito e itálico (usar comandos juntos): ***texto em negrito e itálico***;
- Alterar cor do texto (comando `\textcolor{cor}{texto}`):
  - Exemplo `\textcolor{red}{texto}`: texto vermelho;
  - Exemplo `\textcolor[RGB]{255, 102, 0}`: texto laranja;
  - Exemplo `\textcolor[HTML]{006AD7}`: texto azul;
- Ambiente matemático inline (comando `$ expressão $`): $s = x^2 - 2x + 1$;
- Referência normal (comando `\cite`):
  - (**??**);
  - (**??**);
  - (**??**);
- Referência normal com mais de uma obra (comando `\cite`):
  - (**????**);
  - (**??????**);
- Referência nome e ano (comando `\citeauthorandyear`):
  - **??** (**??**);
  - **??** (**??**);
  - **??** (**??**);

Exemplo 1 de citação direta:

> Os 20 aminoácidos usualmente encontrados como resíduos em proteínas contém um grupo α-carboxil, um grupo α-amino e um grupo R distinto substituído no átomo de carbono α. O átomo de carbono α de todos os aminoácidos, com exceção da glicina, é assimétrico e, portanto, os aminoácidos podem existir em pelo menos duas formas estereoisoméricas. Somente os estereoisômeros L, com uma configuração relacionada à configuração absoluta da molécula de referência L-gliceraldeído, são encontrados em proteínas (**??**, p. 81).

Exemplo 2 de citação direta:

> *These various insecticidal proteins are synthesized during the stationary phase and accumulate in the mother cell as a crystal inclusion which can account for up to 25% of the dry weight of the sporulated cells. The amount of crystal protein produced by a B. thuringiensis culture in laboratory conditions (about 0.5 mg of protein per ml) and the size of the crystals (24) indicate that each cell has to synthesize* 10^6 *to* 2×10^6 *δ-endotoxin molecules during the stationary phase to form a crystal* (**??**, p. 1).

Exemplo de nota de rodapé[1].

[1] Essa é uma nota de rodapé!

3 MATERIAIS E MÉTODOS

A metodologia especifica como os objetivos estabelecidos serão alcançados. As partes constitutivas da metodologia: a amostragem e as formas de coleta, de organização e de análise dos dados.

Deem uma lida no link abaixo para ter uma ideia do que colocar. Conversar com o orientador para definição da metodologia. <https://blog.mettzer.com/metodologia-cientifica/amp/#O_QUE_E_METODOLOGIA_CIENTIFICA>

4 DESCRIÇÃO GERAL DO PRODUTO

Faça uma breve apresentação, visão geral do produto a ser desenvolvido.

?? (**??**);

5 ELICITAÇÃO DE REQUISITOS E ANÁLISE

Parte principal do texto, que contém a exposição ordenada e pormenorizada do assunto. Divide-se em seções e subseções, que variam em função da abordagem do tema e do método.

5.1 Requisitos do Usuário

Texto da justificativa

5.2 Requisitos do Sistema

Fazer uma pequena definição de requisitos do sistema para introduzir a seção (informar a fonte). Assim, esta seção apresenta os requisitos identificados nesse trabalho.

5.2.1 Requisitos Funcionais

Fazer uma pequena definição de requisitos funcionais para introduzir essa seção. Informar quais os atores identificados nessa etapa do desenvolvimento, como o parágrafo a seguir. Terminar o parágrafo informando em qual tabela os requisitos funcionais identificados estão apresentados, mas ou menos como no parágrafo a seguir.

De acordo com **??** (**??**), Requisitos Funcionais são ..... Os atores que interagem com o sistema são: administrador, cliente e fornecedor. Os requisitos funcionais identificados para este sistema estão apresentados na Tabela **??**

Tabela 2 – Requisitos Funcionais

Código	**Descrição**	**Ator**
RF01	O sistema deve permitir a inclusão, alteração e remoção de **estado do país c**om os atributos nome e sigla. Todos os atributos são obrigatórios. Não é permitido incluir 2 ou mais estados com o mesmo nome nem com a mesma sigla.	AtorA
RF02	O sistema deve permitir ...	AtorB
RF03	O sistema deve permitir ...	AtorC
RF04	O sistema deve permitir ...	AtorD

Fonte: Elaborada pelo autor

5.2.2 Requisitos Não-Funcionais

Fazer uma pequena definição de requisitos não-funcionais para introduzir a seção (informar a fonte). Assim, esta seção apresenta os requisitos não-funcionais identificados nesse trabalho, consolidados na Tabela 3

Tabela 3 – Requisitos Não-Funcionais

Código	**Descrição**
RNF01	Os suários deverão operar o sistema após um determinado tempo de treinamento.
RNF02	O sistema deverá ter alta disponibilidade, p.exemplo, 99% do tempo
RNF03	O sistema deverá ser desenvolvido na linguagem NodeJS.
RNF04	O sistema deverá se comunicar com o banco MySQL .

Fonte: Elaborada pelo autor

5.2.3 Restrições, Suposições e Dependências

Forneça uma descrição geral de qualquer item que limitará a decisão do desenvolvedor e uma lista de fatores que afetam os requisitos expressos.

- Item 1;
- Item 2;
- Item n.

5.2.4 Requisitos Adiados

Forneça uma descrição dos requisitos identificados pelo usuário ou pelo desenvolvedor, mas que não serão desenvolvidos nessa primeira iteração, ou seja, serão analisados e desenvolvidos posteriormente.

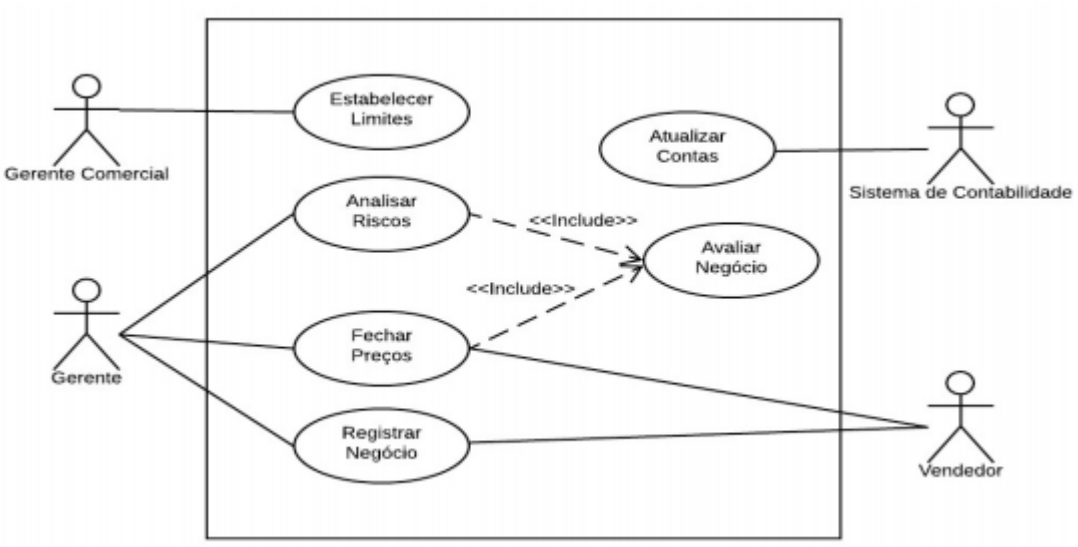
5.3 Casos de Uso

Forneça uma breve definição de casos de uso (com referência da fonte) e informe que os casos de uso identificados neste trabalho estão aqui apresentados.

5.3.1 Diagrama de Casos de Uso

Forneça uma breve definição de diagrama de casos de uso (referência da fonte). O diagrama de casos de uso elaborado neste trabalho está apresentado na Figura 2.

Figura 2 – Diagrama de Casos de Uso



Fonte: Elaborada pelo autor

5.3.2 Especificação dos Casos de Uso

Uma especificação de caso de uso fornece detalhe textual para um caso de uso. Uma estrutura de tópicos de amostra de uma especificação de caso de uso é fornecida. É possível reutilizar e modificar a estrutura de tópicos conforme necessário em um documento de especificação de caso de uso. A Tabela 4 apresenta um modelo para especificação de casos de uso (extraído de IBM-Especificação de Casos de Uso).

Tabela 4 – Modelo para Especificação de Casos de Uso

Item	**Descrição**
Nome do Caso de Uso	Declara o nome do caso de uso. Geralmente, o nome expressa o objetivo ou resultado observável do caso de uso, como "Sacar Dinheiro"no caso de um caixa eletrônico.
Descrição Resumida	Descreve a função e o objetivo do caso de uso.
Fluxo de Eventos	Apresenta o fluxo básico e os fluxos alternativos. O fluxo de eventos descreve o comportamento do sistema; ele não descreve como o sistema funciona, os detalhes da apresentação ou os detalhes da interface com o usuário. Se forem trocadas informações, o caso de uso deverá ser específico sobre o que será transmitido de um lado para outro. Por exemplo, em vez de descrever uma ação como "o agente insere informações do cliente", indique que "o agente insere o nome e o endereço do cliente".
Fluxo Básico	Descreve o comportamento principal ideal do sistema.
Fluxos Alternativos	Descreve exceções ou desvios do fluxo básico, como o comportamento do sistema quando o agente insere um ID de usuário incorreto e a autenticação do usuário falha.
Requisitos Especiais	Requisitos não-funcionais que são específicos para um caso de uso mas que não são especificados no texto do fluxo do caso de uso de eventos. Exemplos de requisitos especiais incluem estes fatores: requisitos legais e regulamentares; padrões de aplicativo; atributos de qualidade do sistema, incluindo usabilidade, confiabilidade, desempenho e capacidade de suporte; sistemas operacionais e ambientes; requisitos de compatibilidade e restrições de design.
Condições Prévias	Um estado do sistema que deve estar presente antes de um caso de uso ser iniciado.
Pós-Condições	Uma lista de estados possíveis para o sistema imediatamente após a conclusão de um caso de uso.
Pontos de Extensão	Um ponto no fluxo de eventos do caso de uso em que outro caso de uso é referenciado.

Fonte: Extraído de IBM-Especificação de Casos de Uso

Elaborar as especificações para cada caso de uso, quando for necessário, como por exemplo: A especificação do caso de uso Finalizar Compra é

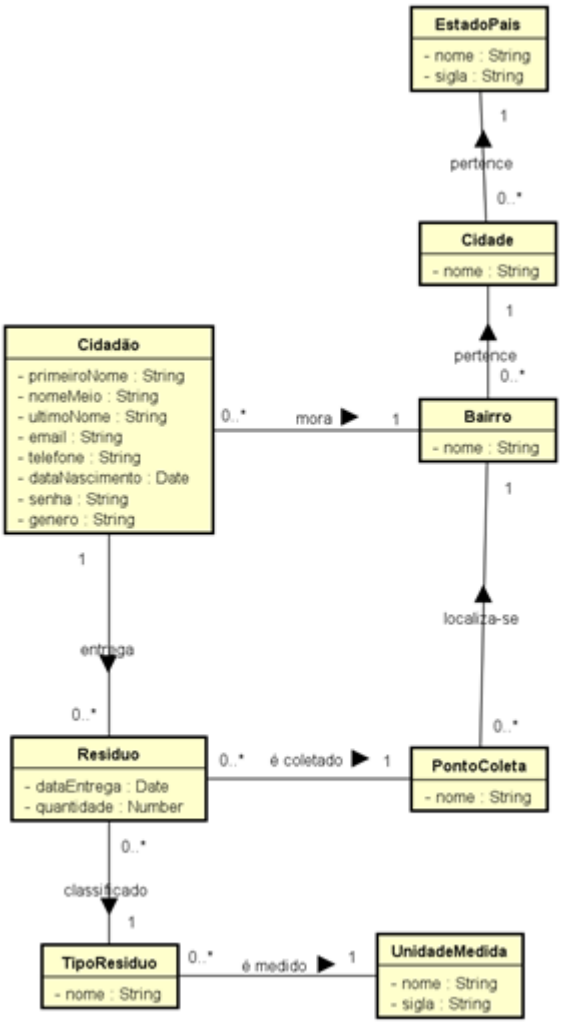
5.4 Modelo de Domínio

Fazer uma pequena definição de modelo de domínio para introduzir essa seção. Terminar o parágrafo informando qual figura apresenta o modelo de domínio. Elabore o diagrama de classes somente com os atributos, para auxiliar no entendimento do domínio da aplicação, conforme Figura 3.

De acordo com **??** (**??**), o modelo de domínio é ..... . A Figura 3 apresenta o modelo de domínio desenvolvido neste trabalho.

Caso de Uso Finalizar Comprar
Referências RF02
Descrição Geral O caso de uso inicia-se quando o cliente deseja efetuar compra dos produtos que estão inseridos no carrinho de compras.
Atores Cliente
Pré-Condições Cliente logado no sistema, produtos já inseridos no carrinho de compras.
Garantia de Sucesso (Pós-Condições Pedido fechado, compra efetuada, sistema aguardando confirmação de pagamento.
Requisitos Especiais
Fluxo Básico 1. Cliente deseja finalizar compra, sistema solicita que informe a forma de pagamento e de entrega; 2. Cliente deseja efetuar pagamento em forma de cartão de crédito/débito; 3. Sistema solicita informações do cartão do cliente; 4. Sistema faz validação das informações; 5. Sistema gera o número do pedido; 6. Compra finalizada com sucesso.
Fluxo Alternativo 1. Cliente deseja efetuar pagamento através de boleto bancário 1.1 Sistema gera boleto para o cliente; 1.2 Retorne ao passo 5.

Figura 3 – Modelo de Domínio



Fonte: Elaborada pelo autor

5.5 Diagrama de Objetos

Fazer uma pequena definição de diagrama de objetos para introduzir essa seção. Terminar o parágrafo informando qual figura apresenta o diagrama de objetos. Elabore o diagrama de objetos para cada classe identificada no modelo de domínio.

5.6 Diagrama de Classes de Análise

Fazer uma pequena definição de diagrama de classes para introduzir essa seção. Terminar o parágrafo informando qual figura apresenta o diagrama de classes. É uma evolução do modelo de domínio, elabore o diagrama de classes com os atributos e métodos.

5.7 Diagrama de Atividades

Um diagrama de atividade ilustra a natureza dinâmica de um sistema pela modelagem do fluxo de controle de atividade à atividade. Uma atividade representa uma operação em alguma classe no sistema que resulta em uma mudança no estado do sistema.

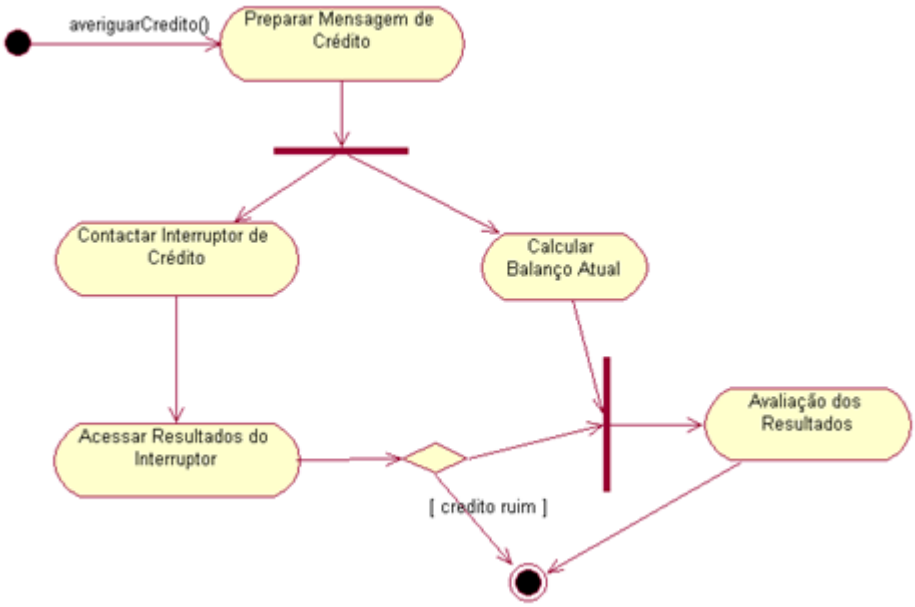
Tipicamente, diagramas de atividades são usados para modelar fluxos de processos, processos de negócios ou operações internas. o diagrama de atividades é similar a uma máquina de estados, mas tem um propósito diferente, o qual envolve capturar ações e seus resultados em termos de mudanças do estado do objeto.

O diagrama de atividades é representado por um gráfico de atividades que mostram o fluxo de uma atividade para outra. Esse fluxo é mostrado através de transições, que são setas direcionadas, mostrando o caminho entre os estados de atividade (ação).

A Figura 4 mostra o diagrama de atividades para a operação averiguarCredito na classe Pedido da Virtual LTDA. Note que a atividade "Preparar Mensagem de Credito" define o que fazer, mas não como fazer (RIBEIRO, 2021).

Um diagrama de atividades é normalmente composto pelos seguintes elementos: atividades (ações), estados de atividade (ação), transição, fluxo de objeto, estado inicial, estado final, branching, sincronização, raias.

Figura 4 – Diagrama de Atividades

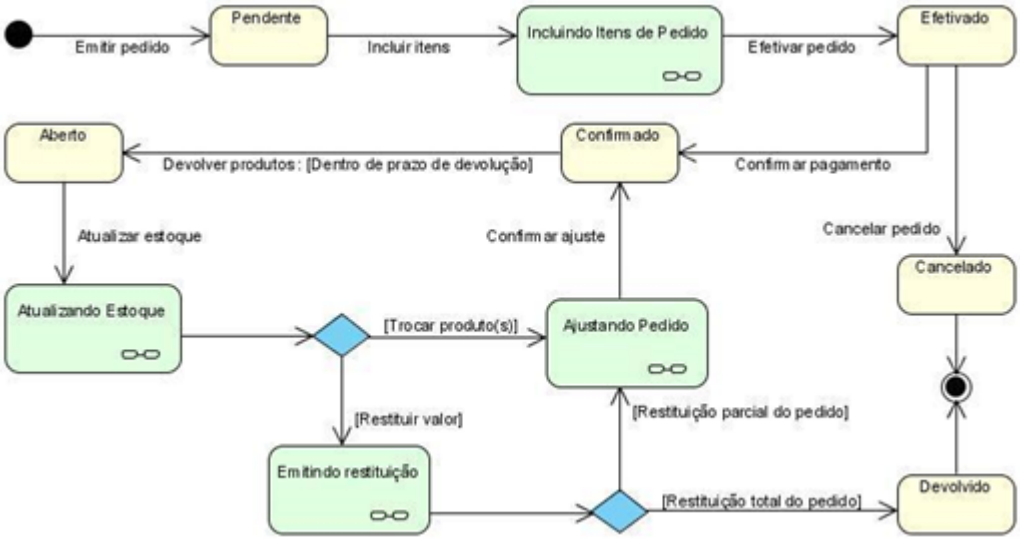


Fonte: Elaborada pelo autor

5.8 Diagrama de Estados

Fazer uma introdução mais ou menos como o parágrafo a seguir. Um diagrama de estados mostra os possíveis estados de um objeto e as transações responsáveis pelas suas mudanças de estado, conforme exemplo apresentado na Figura 5.

Figura 5 – Diagrama de Estados



Fonte: Elaborada pelo autor

6 PROJETO DE SOFTWARE

Parte principal do texto, que contém a exposição ordenada e pormenorizada do assunto. Divide-se em seções e subseções, que variam em função da abordagem do tema e do método.

6.1 Projeto de Interface

Apresentar o protótipo das interfaces do sistema. Podem ser construídas utilizando qualquer ferramenta. Apresente os padrões utilizados.

6.2 Projeto de Dados

Informar nesta seção qual o Banco de Dados a ser utilizado, qual a IDE utilizada para manipulação do Banco de Dados, assim como descrever quais os tipos de dados aceitos no Banco de Dados.

6.2.1 Mapeamento Objeto-Relacional

Uma vez que foi elaborada uma modelagem orientada a objetos, utilizando o diagrama de classes UML, e o banco de dados a ser utilizado é relacional, deve-se identificar as relações. Assim, a partir dos requisitos e do modelo de domínio enumerar as relações que devem formar o banco de dados.

Regras para realizar o mapeamento: (Helen, colocar as regras aqui para que os alunos escrevam)

- Item 1;
- Item 2;
- Item n.

6.2.2 Estrutura das Tabelas no Banco de Dados

Explicar qual o padrão adotado para o nome dos objetos de banco de dados, como por nome da chave primária, chave estrangeira, das constraints e das chaves únicas. Por exemplo: foi convencionado que o nome dos objetos devem obedecer o que está definido na Tabela 5.

Tabela 5 – Convenção para Nome dos Objetos no Banco de Dados

Objeto	**Padrão Adotado**
Chave Primária	NomeDaTabela_PK
Chave Estrangeira	NomeDaTabela_NomeDaTabelaEstrangeira_FK_nn, onde nn é a sequência de ocorrência do par NomeDaTabela e NomeDaTabelaEstrangeira
Check	NomeDaTabela_CK_nn, onde nn é a sequência de checks da tabela
Chave Única	NomeDaTabela_UK_nn, onde nn é a sequência de chave única da tabela

Fonte: Elaborada pelo autor

Para melhor compreensão, as tabelas do banco de dados propostas neste trabalho estão consolidadas na Tabela 6, as quais estão individualmente detalhadas.

Tabela 6 – Tabelas Identificadas neste Trabalho

Tabela do Banco de Dados	**Tabela no Documento**
Estado	Tabela 7
Cidade	Tabela 8
NomeDaTabelann	Tabela 9

Fonte: Elaborada pelo autor

Preencher o quadro a seguir para cada uma das relações identificadas no item Mapeamento OO-Relacional, que deve refletir as características das tabelas a serem criadas no banco de dados. Para melhor compreensão sobre qual a estrutura física das tabelas, seguem dois exemplos: a Tabela 6 é um exemplo da estrutura de dados Estado e a Tabela 6 é um exemplo da estrutura de dados Cidade.

Tabela 7 – Estado

Campo	**Tipo de Dado**	**Obrigatório?**	**Chave Primária?**	**Tabela**	**Campo**	**Grupo**
id	**Number**	**X**	**X**			
nome	**Varchar2(100)**	**X**				**1**
sigla	**Varchar2(2)**	**X**				**2**

Fonte: Elaborada pelo autor

INSTRUÇÃO: Grupo é a chave única que você deseja criar (UK_01, UK_02) Ordem é a sequência dos campos que devem compor a chave única, quando existir mais de um campo na sua chave única

Tabela 8 – Cidade

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo
id	**Number**	**X**	**X**			
nome	**Varchar2(200)**	**X**				**1**
idEstado	**Number**	**X**		**Estado**	**id**	**1**
latitude	**Number**	**X**				**2**
longitude	**Number**	**X**				**2**

Fonte: Elaborada pelo autor

Tabela 9 – NomeDaTabelaNN

Campo	Tipo de Dado	Obrigatório?	Chave Primária?	Tabela	Campo	Grupo

Fonte: Elaborada pelo autor

6.2.3 Diagrama de Pacotes

Escreva aqui.

6.2.4 Diagrama de Classes de Projeto

Apresentar as classes de projeto identificadas. Alguns materiais para leitura extra sobre classes de projeto podem ser lidos em: <http://www.ic.uff.br/~anselmo/cursos/ProjSoft/apresentacoes/Projeto%20detalhado%20-%20Classes.pdf> <https://edisciplinas.usp.br/pluginfile.php/383727/mod_resource/content/2/Aula07_DiagramaDeClasse.pdf> <http://www.dsc.ufcg.edu.br/~jacques/cursos/apoo/html/proj1/proj8.htm>

6.3 Projeto Procedimental

Projetar Software é o processo de aplicar várias técnicas e princípios com o propósito de se definir um dispositivo, processo ou sistema, com detalhes suficientes para permitir sua realização física (Taylor-59).

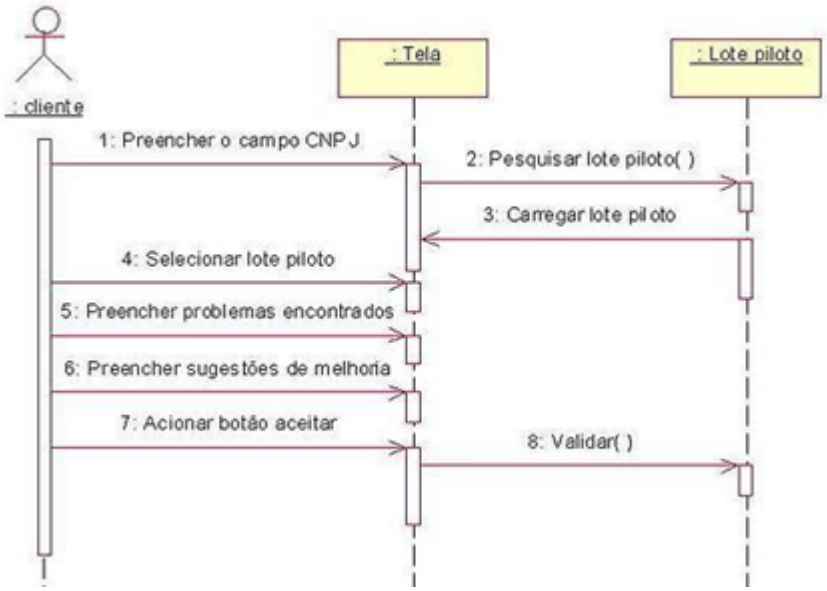
O Projeto de software é o núcleo técnico da Engenharia de Software. É a única maneira de se traduzir "com precisão", os requisitos do usuário para um produto ou sistema acabado. Meta: Traduzir requisitos numa representação de software (Portella).

6.3.1 Diagrama de Sequência

Os diagramas de sequência modelam as interações entre objetos em um único caso de uso e ilustram como as diferentes partes de um sistema interagem entre si para realizar

uma função e a ordem em que as interações ocorrem quando um caso de uso é executado (Creately, s.d.), conforme apresentado na Figura 6.

Figura 6 – Diagrama de Sequência



Fonte: Elaborada pelo autor

6.3.2 Diagrama de Atividades

Havendo necessidade de detalhamento de algum procedimento, este pode ser apresentado na forma de um diagrama de atividades o qual deve ser inserido nessa seção. Podem ser elaborados quantos diagramas de atividades se fizerem necessários.

6.3.3 Diagrama de Estados

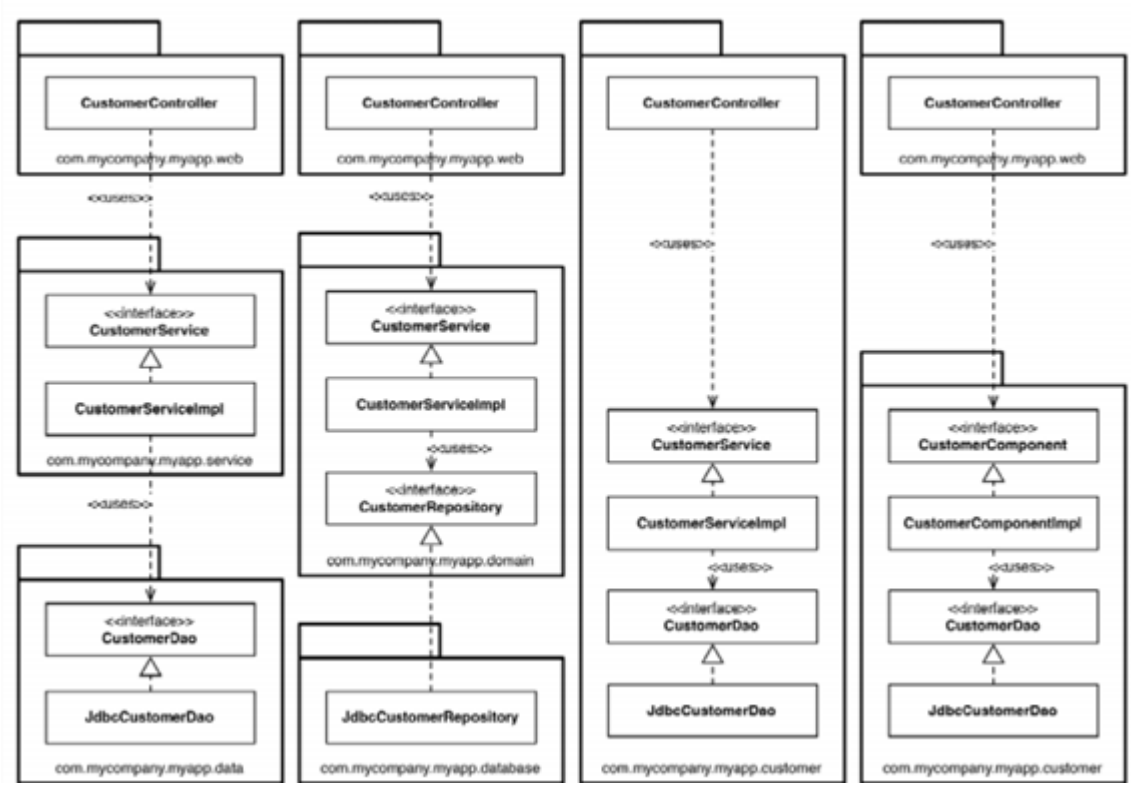
Sendo necessária a elaboração do diagrama de estados na fase de projeto, os novos diagramas devem ser apresentados nessa seção.

6.4 Projeto Arquitetural

O projeto arquitetural precede a etapa de construção da obra. O projeto arquitetural determina as partes de uma construção e como estas devem interagir. A arquitetura garante a unidade da obra, ou seja, a consistência entre as suas partes (Vergilio).

Ver algumas definições em (Silva), sendo que um exemplo está apresentado na Figura 7.

Figura 7 – Diferentes Detalhamentos dos Serviços Relacionados ao Cliente



Fonte: Elaborada pelo autor

7 IMPLEMENTAÇÃO

Descrever nesse capítulo as características do ambiente de desenvolvimento, servidor de aplicação, servidor de dados, pacotes utilizados etc etc etc.

8 TESTE

Descrever nesse capítulo quais e como foram os testes realizados. Os testes podem ser apresentados na forma de casos de teste. Um caso de teste consiste em conjunto de detalhes necessários para se realizar um teste de software.

A seguir encontra-se um modelo para especificação de um caso de teste (CEDRO, 2021):

Título O título do caso de teste deverá ser sucinto, simples e autoexplicativo com informações para que o Analista de Teste saiba a validação a qual o teste se propõe. Exemplos: • Validar upload de arquivo; • Validar cadastro de usuário com perfil administrador; • Validar envio de ordem de compra.
Objetivo O objetivo do caso de teste é descrever o que será executado, fornecendo uma visão geral do teste que será realizado. Exemplos: • Verificar se realiza o upload do arquivo com as extensões permitidas; • Verificar se o cadastro é efetivado após preencher as informações corretamente; • Verificar se a ordem de compra é enviada informando o ativo, quantidade e preço.
Pré-Condição São condições necessárias para que o caso de teste consiga ser executado. Evitar que não tenha alguma informação necessária (Exemplo: solicitar a edição de um usuário em específico e na pré-condição não informar que o usuário deve estar cadastrado). Exemplos: • Usuário cadastrado e autenticado no sistema; • Ordem de compra enviada e executada; • Usuário com perfil Administrador.

Passos Os passos são necessários para descrever todas as ações que o analista deve seguir durante a execução para chegar ao resultado esperado. Devendo iniciar com um verbo infinitivo (acessar, preencher, clicar, verificar) ou imperativo (acesse, preencha, clique, verifique). Exemplos: • Acessar a tela Negociação > Boleta; • Clicar no botão "Entrar"; • Verificar se a edição foi salva no banco de dados; • Preencher os campos do cadastro.
Resultados Esperados Descrever o comportamento esperado do sistema após executar os passos detalhados. Informar os verbos no presente (valida, apresenta, recupera, retorna). Evitar frases como "O sistema **deve** retornar a mensagem. . . ", prefira usar "O sistema retorna a mensagem. . . " para não deixar nenhuma dúvida do resultado esperado. Exemplos: • Sistema apresenta a tela de edição com os campos preenchidos.; • A ordem é enviada e executada com o preço informado; • O cadastro é salvo no banco de dados.

Os casos de teste podem ser especificados usando uma ferramenta de software. Caso este seja o caso, esta seção deve apresentar qual a ferramenta e foi utilizada no desenvolvimento deste trabalho. A Figura 8 apresenta um exemplo de caso de teste especificado usando a ferramenta Testlink.

Figura 8 – Exemplo de Caso de Teste Elaborado na Ferramenta Testlink

Caso de Teste

CT-AUT-140:Validar cadastro de cliente

Versão 1

Objetivo do Teste:

Verificar se realiza o cadastro do cliente informando nome, CPF e telefone.

Pré-condições

1. Usuário cadastrado e autenticado no Portal ABC;
2. Usuário com perfil Administrador;
3. Possuir CPF válido.

#	Ações do Passo	Resultados Esperados:
1	1 - Acessar a tela de cadastro de cliente no Portal ABC: Menu principal > Cadastros > Cliente; 2 - Preencher os campos com dados válidos: • Nome • CPF • Telefone 3 - Clicar em Salvar; 4 - Verifique se o cadastro do cliente foi salvo no Banco de dados.	1 - Sistema exibe a tela de cadastro de cliente com os campos vazios; 2 - Após salvar o cadastro exibe a mensagem de sucesso: "Cliente cadastrado."; 3 - O registro do cliente é salvo no Banco de dados.

Fonte: Elaborada pelo autor

9 IMPLANTAÇÃO

Em qual servidor o sistema está implantado, em qual servidor está a aplicação e em qual servidor está o banco de dados, se foi feito registro de domínio, como deve ser feita a implantação e configuração do sistema, etc etc etc.

Caso o sistema esteja em execução em um ou mais clientes, apresentar nesse capítulo como foi a implantação. Sendo necessário, podem ser usados diagramas UML, como o diagrama de atividades, para apresentar as tarefas de implantação.

10 RESULTADOS E DISCUSSÃO

Texto dos resultados.

11 TRABALHOS RELACIONADOS

Texto dos trabalhos relacionados.

12 CONCLUSÕES/CONCLUSÕES PARCIAIS

Texto das conclusões.

Obs: Este capítulo deve ser intitulado “Conclusões Parciais” em trabalhos de graduação para a Validação de Projeto de TCC. Na Avaliação Final de TCC o nome do capítulo deve ser “Conclusões”.

13 CRONOGRAMA

Segue abaixo o cronograma das atividades que serão executadas até a Avaliação Final de TCC.

Obs: Para facilitar, crie o cronograma usando o modelo do Word contido no projeto (imagens/templateCronograma.docx), ou qualquer outro *software*, salve a imagem e atualize o arquivo imagens/cronograma.png.

		Meses											
		JAN	FEV	MAR	ABR	MAI	JUN	JUL	AGO	SET	OUT	NOV	DEZ
Atividades	1	X	X										
	2		X	X	X								
	3			X	X	X							
	4			X	X								
	5	X	X	X	X	X	X	X	X	X	X	X	X

1. Descrição da atividade 1;
2. Descrição da atividade 2;
3. Descrição da atividade 3;
4. Descrição da atividade 4;
5. Descrição da atividade 5.

Obs: Este capítulo deve ser elaborado e estar contido em trabalhos de graduação para a Validação de Projeto de TCC. Na Avaliação Final de TCC este capítulo não deve existir, visto que não haverá atividades após a Avaliação Final.

REFERÊNCIAS

BORODIN, A.; CASHMAN, D.; MAGEN, A. How well can primal-dual and local-ratio algorithms perform? **ACM Trans. Algorithms**, Association for Computing Machinery, New York, NY, USA, v. 7, n. 3, jul. 2011. ISSN 1549-6325. Disponível em: <https://doi-org.ez338.periodicos.capes.gov.br/10.1145/1978782.1978784>. Citado na página 21.

CORMEN, T. H. et al. **Introduction to Algorithms**. 2nd. ed. The MIT Press, 2001. ISBN 0262032937. Disponível em: <http://www.amazon.com/Introduction-Algorithms-Thomas-H-Cormen/dp/0262032937%3FSubscriptionId%3D13CT5CVB80YFWJEPWS02%26tag%3Dws%26linkCode%3Dxm2%26camp%3D2025%26creative%3D165953%26creativeASIN%3D0262032937>. Citado na página 20.

JUKNA, S. et al. On p versus np ∩ co-np for decision trees and read-once branching programs. **Computational Complexity**, Springer, v. 8, n. 4, p. 357–370, 1999. Citado na página 21.

KUMAR, V. Algorithms for constraint-satisfaction problems: A survey. **AI Magazine**, v. 13, n. 1, p. 32, Mar. 1992. Disponível em: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/976>. Citado 2 vezes nas páginas 19 e 20.

MACKWORTH, A. K. Consistency in networks of relations. **Artificial Intelligence**, v. 8, n. 1, p. 99–118, 1977. ISSN 0004-3702. Disponível em: <https://www.sciencedirect.com/science/article/pii/0004370277900078>. Citado na página 20.

PAPADIMITRIOU, C. H.; YANNAKAKIS, M. On limited nondeterminism and the complexity of the v-c dimension. **Journal of Computer and System Sciences**, v. 53, n. 2, p. 161–170, 1996. ISSN 0022-0000. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0022000096900586>. Citado na página 20.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 3. ed. [S.l.]: Prentice Hall, 2010. Citado 4 vezes nas páginas 19, 20, 21 e 22.

SIPSER, M. **Introduction to the Theory of Computation**. 3. ed. [S.l.]: Cengage Learning, 2012. Citado 2 vezes nas páginas 19 e 20.

APÊNDICE A – TÍTULO DO APÊNDICE A

Texto do Apêndice A.

APÊNDICE B – TÍTULO DO APÊNDICE B

Texto do Apêndice B.

APÊNDICE C – TÍTULO DO APÊNDICE C

Texto do Apêndice C.

APÊNDICE D – TÍTULO DO APÊNDICE D

Texto do Apêndice D.

APÊNDICE E – TÍTULO DO APÊNDICE E

Texto do Apêndice E.

ANEXO A – TÍTULO DO ANEXO A

Texto do Anexo A.

ANEXO B – TÍTULO DO ANEXO B

Texto do Anexo B.

ANEXO C – TÍTULO DO ANEXO C

Texto do Anexo C.

ANEXO D – TÍTULO DO ANEXO D

Texto do Anexo D.

ANEXO E – TÍTULO DO ANEXO E

Texto do Anexo E.